



工程的目录结构

一个工程应该划分为若干个目录，目录应该是模块划分的体现。如：

```
Project    →ModuleName1
           ModuleName2
           .....
           ModuleNameN
           Include
           Bin
           Lib
```

其中 include 目录下应该放置公共的头文件。

Bin 目录下放执行文件

Lib 目录下*.lib 文件

- 目录名称要有意义

文件

文件的结构

- 文件的种类有两种：源文件 (*.c) 和头文件 (*.h)。
- 文件划分的准则
 - 高内聚；
 - 低耦合；

文件的命名及文件的长度

- 文件名只能由小写字母、数字和 '_' 组合而成。
- 使用英语命名，尽量选用可以发音和有意义的名字。
- 一个文件的长度不要超过 1000 行代码。

源文件

- 一般说来，源文件的整体构成包括以下内容（有时可能不存在某些部分）。具体情况请参见源文件模板。

源文件的注释框

包含文件部分

宏定义部分

类型定义部分

结构体定义部分

全局变量定义部分



文件 *static* 变量定义部分
函数原型声明
调试开关定义部分
函数定义部分

头文件

- 一般说来，头文件的整体构成包括以下内容（有时可能不存在某些部分）。具体情况请参见头文件模板。

头文件的注释框
包含文件部分
宏定义部分
类型定义部分
结构体/类定义部分
函数原型声明

- 头文件应采用 `#ifndef/#define/#endif` 的方式来防止多次被包含，其中使用的宏名为：“__” + “文件名” + “_” + “扩展名”，文件名与扩展名均为大写。对
- 头文件的包含应该采用相对路径，而不采用绝对路径。
- `#include` 包含的文件名称应由小写字母、数字和 ‘_’ 组合而成。
- 为了减少执行程序的大小，不应包含不需要的头文件。
- 头文件中不要定义变量。



程序格式一排版、注释

排版

分行

- 较长的语句、表达式等要分成多行书写，一程序不要超过 81 列；
- 不允许把多个短语句写在一行中，即一行只写一条语句。
- 循环、判断等语句中若有较长的表达式或语句，则要进行适应的划分；划分的原则：整齐、美观、符合其他编码规范。
- 若函数的参数较长，则要进行适当的划分。划分的原则：整齐、美观、符合其他编码规范。
- 在两个函数定义之间必须使用空行分开。
- 在函数定义中，用空行将代码按逻辑片断划分。
例如两个并列的判断语句之间就可以用空行来区分；变量的定义、变量的初始化以及真正的处理语句之间就可以用空行来区分。
- 意义不一致的宏定义之间、全局变量之间、函数声明之间需要添加空行。
- 如果函数参数不只有一个，则每个参数的定义各占一行，每行前有一个制表符的缩进。
- 最后一个参数的定义后要紧跟右括号。
- 定义函数体的左右大括号要各自独占一行。【可检查： 04-04-01】

空格

- 单目操作符后不要加空格。
- 单目操作符前不要加空格。
- 赋值符号前后要有一个空格。
位逻辑运算符后的 ASCII 码应该有一个空格。
- 在条件表达式与括号间应该有一个空格。
- 在问号表达式（？）前后应该有一个空格。
- 在二元逻辑操作符前后应该有一个空格。
- 在比较操作符前后应该有一个空格。
- 在“.”、“->”前后不应该有空格。
- 在数组名与“[”之间不应该有空格。
- “,”之后应该有空格。
- “;”之后应该有空格。
- “sizeof”与“(”之间不应该有空格；“(”与类型之间不应该有空格。



缩进

- 代码缩进要使用制表符，而不是直接使用空格。
- 对嵌套语句使用一个制表符----Tab（4 个字符）的缩进。
- 尽可能保证缩进嵌套的层数不超过四层。

注释

- 边写代码边注释，修改代码同时修改相应的注释，以保证注释与代码的一致性。
- 注释写在说明代码的上面。
- 注释的目的是解释代码的目的、功能和采用的方法，提供代码以外的信息，帮助读者理解代码，防止没必要的重复注释信息。
- 注释与前面的执行语句之间空一行。
- 源文件的开头要有注释，但每个业务/项目由于客户的要求不同而有不同的注释格式，这里对注释格式不作统一要求。
- 注释不要有嵌套。
- 应对不易理解的分枝条件表达式加注释。
- 不易理解的循环，应说明出口条件。

函数

函数命名

- 函数名只能由大小写字母与数字构成。
- 函数名中除前缀外的各个单词首字母要大写。

函数说明

- 函数必须都有函数说明。
说明中必须包含如下内容：
 1. 名称 (Function Name)：函数的名称。
 2. 描述 (Description)：函数功能的简单描述。
 3. 作成日期 (Created)：格式为 YY/MM/DD 的日期，如 03/09/28。
 4. 参数 (Parameter)：函数参数的说明。
 - a. 参数说明的顺序依次为参数名称、参数类型、输入参数还是输出参数。
 - b. 如果函数没有任何参数，本项填写“VOID”。



5. 返回值类型(Return Code)：函数返回值的类型。
 - a. 如果函数不需要返回值，本项填写“VOID”。
6. 作者(Author)：函数的作者。

函数声明

- 一定要显示声明函数的返回值类型。
- 如果函数不需要返回值，要声明为 void 类型。
- 如果函数没有参数，参数处要书写 void。
- 函数的声明要与函数定义一致（包括函数的类型及参数的类型、个数、顺序）。

函数定义

- 注意控制参数的数量，一般来说不要超过 7 个，当参数过多时，应该考虑将参数定义为一个结构体，并且将结构体指针作为参数。
- 函数的大小不要过长，一般定为 60 行以内（除去注释，空行，变量定义，调试开关等）。
- 一个函数仅完成一件功能。
- 防止将函数的参数作为工作变量。
- 编写可重入函数时，若使用全局变量，则应通过信号量保护。对于可能在中断处理程序中访问的全局变量，应该用关中断的方式进行保护。

函数调用

- 回避函数的递归调用。
- 调用没有参数的函数时、不要忘了加括弧“()”。
- 使用从 pointer 型函数返回值的时候, 要确认它不是 NULL 之后再使用。
- 使用标准 Library 的函数的时候、在调用之前要确认，成为参数的值是否是作为其函数的参数能够取得的范围内的值。
- 函数调用传递的参数（类型、个数、顺序要与函数定义一致）。



变量

变量定义与初始化

- 关于变量的定义：在 C 语言中，变量应该在函数开头定义，变量的命名遵循标识符的命名原则。
- 原则上在定义变量的同时对其初始化，而不是使用前再初始化
- 一行只作一个变量定义。但是象坐标 x、y、z，纬度、经度等对同时进行定义有非常重要意义的在一行可进行多个变量定义。
- 要防止局部变量与全局变量同名。
- 严禁读取未经初始化的变量。
- 不要定义过大的局部变量，以免堆栈溢出，可以定义成 static 全局变量或从内存池中分配。

变量的类型转换

- 不提倡变量间的类型转换。
- 如需要类型转换，请采用显形方式进行。
- 建议不要直接比较 BOOL 类型变量。
- 不要将有符号数强制转化为无符号数。
- 不要将浮点型指针强制转化为整型指针。

变量使用的注意事项

- 不要返回局部变量的地址。
- 用 unsigned 定义不可能为负数的变量。
- 变量的使用目的只能有一个，禁止以多个目的使用变量。
- 函数中避免存在不使用的变量。



编码规范示例：

```

/*****/
/*名称：文件名称
/*描述：描述文件实现的功能
/*作成日期：
/*参数：
    参数 1：参数名称、参数类型、输入/输出参数、参数含义
    参数 2：参数名称、参数类型、输入/输出参数、参数含义
    参数 3：参数名称、参数类型、输入/输出参数、参数含义

/*返回值：返回值名称、类型、含义
    如果函数不需要返回值，本项填写“VOID”。
/*作者：
/*****/

#include<gtk/gtk.h>

/*****/
/*名称：函数名称
/*描述：描述函数的功能
/*作成日期：
/*参数：
    参数 1：参数名称、参数类型、输入/输出参数、参数含义
    参数 2：参数名称、参数类型、输入/输出参数、参数含义
    参数 3：参数名称、参数类型、输入/输出参数、参数含义
/*返回值：返回值名称、类型、含义
    如果函数不需要返回值，本项填写“VOID”。
/*作者：
/*****/

void cb_button(GtkWidget *widget, gpointer data)
{
    // 按钮"button"的回调函数
    gtk_main_quit();
}

/*****/
```



```
/*名称：函数名称
/*描述：描述函数功能
/*作成日期：
/*参数：参数 1：参数名称、参数类型、输入/输出参数、参数含义
        参数 2：参数名称、参数类型、输入/输出参数、参数含义
        参数 3：参数名称、参数类型、输入/输出参数、参数含义
/*返回值：返回值名称、类型、含义
        如果函数不需要返回值，本项填写“VOID”。
/*作者：
/*****/
int main(int argc, char *argv[])
{
    //主窗口对象
    GtkWidget *main_window;

    //将要放置到主窗口中的按钮对象
    GtkWidget *button;

    gtk_init(&argc, &argv);
    main_window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
    gtk_window_set_title(GTK_WINDOW(main_window), "Hello, Dubuntu2!");

    //设置窗口的默认大小（宽 200，高度 50）
    gtk_window_set_default_size(GTK_WINDOW(main_window), 200, 50);

    button = gtk_button_new_with_label("退出程序");
    gtk_container_add(GTK_CONTAINER(main_window), button);

    //为"button"连接“单击事件”要调用的回调函数
    g_signal_connect(G_OBJECT(button), "clicked", G_CALLBACK(cb_button), NULL);

    //上边的两句可以合为 gtk_widget_show_all(window)
    gtk_widget_show(button);
    gtk_widget_show(main_window);

    gtk_main();
    return 0;
}
```